

TaskWeaver Modeling Language

Final presentation of our language in the 'Metamodeling' course

University of Fribourg, 10.12.2025

Motivation

What is missing today?

-> Automation workflows are:

- Tool-specific (Airflow, Zapier, Node-RED)
- Hard to conceptually model
- Not designed for data + execution together

-> Existing BPMN:

- Sometimes too generic
- No explicit data processing semantics
- No built-in automation execution logic



Research Gap

No lightweight, visual modeling
language focused on **automation +
data pipelines + execution.**



General Idea

What is TaskWeaver?



A visual workflow modeling language for:

Data
ingestion

Automation

Script
execution

API calls

Database
interaction



-> It also contains good visual notations for data-driven automation workflows, data-pipeline workflows, service-oriented automation, lightweight process automation and educational workflows.

Benefits

Expected Benefits:

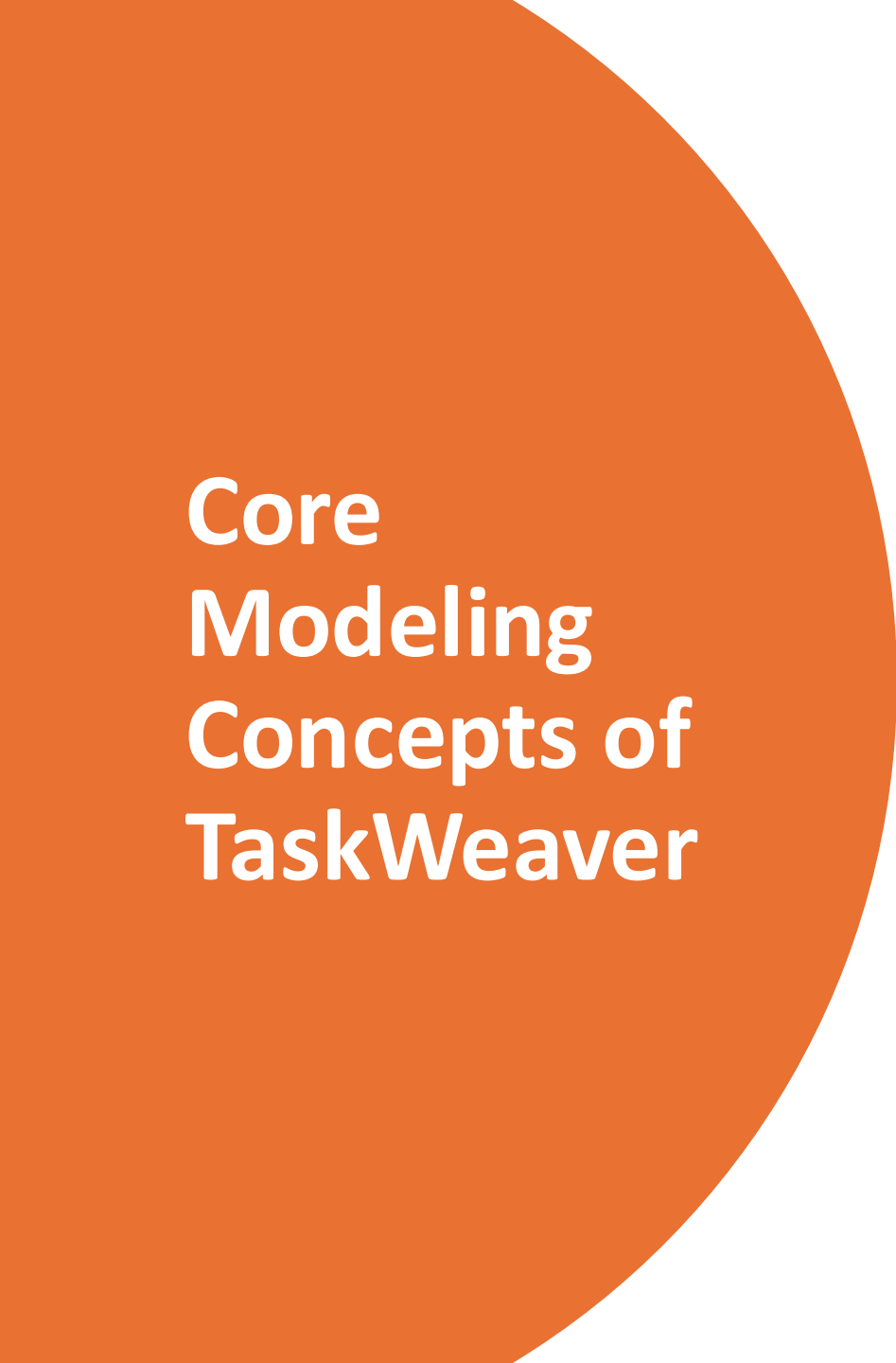
-> Clear separation of:

- Control flow (SequenceFlow)
- Data flow (DataFlow)

-> Visual + Executable

-> Platform-independent modeling

-> Automatic validation & simulation via ADOxx

A large orange circle on the left side of the slide, partially cut off by the edge.

Core Modeling Concepts of TaskWeaver

Event → Start, End, Error

Task → Execution unit

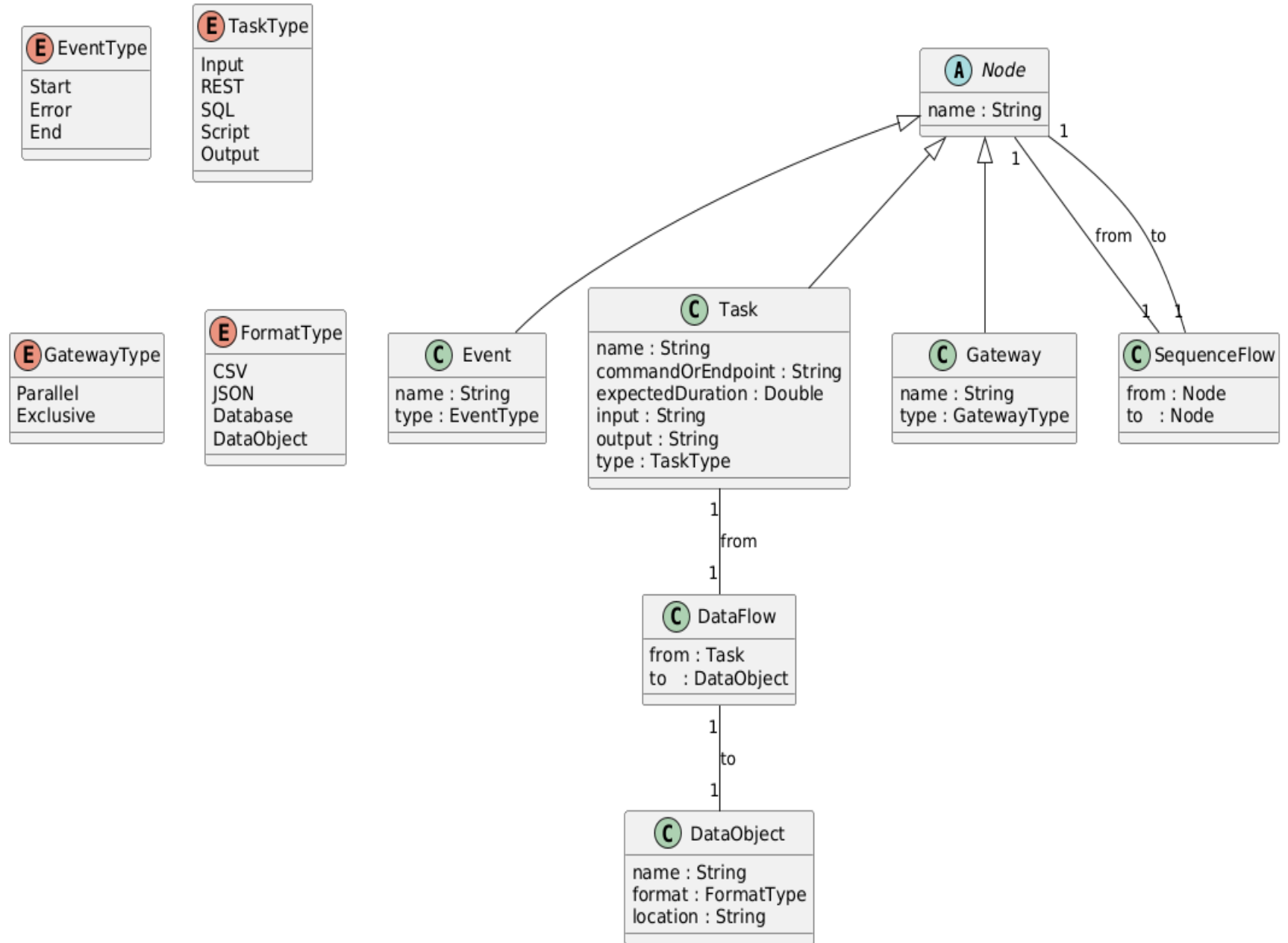
Gateway → Parallel / Exclusive

DataObject → CSV, JSON, Database

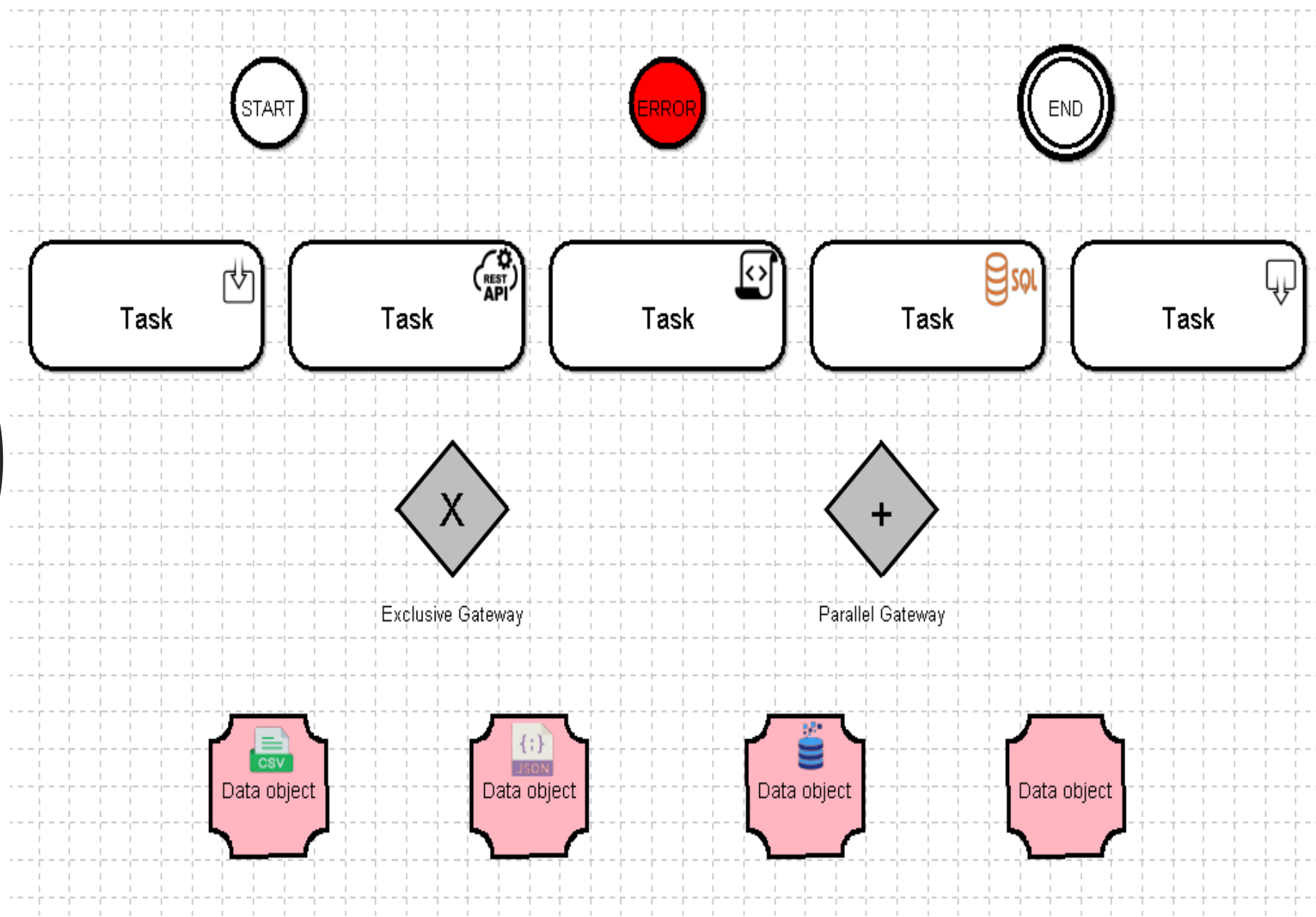
SequenceFlow → Control flow

DataFlow → Data dependency

TW Metamodel Overview



Graphical Notations



FDMM Formalization

Metamodel definition, object and data types

DEFINITION: $MT_TaskWeaver = \langle O_TaskWeaver, D_TaskWeaver, A_TaskWeaver \rangle$

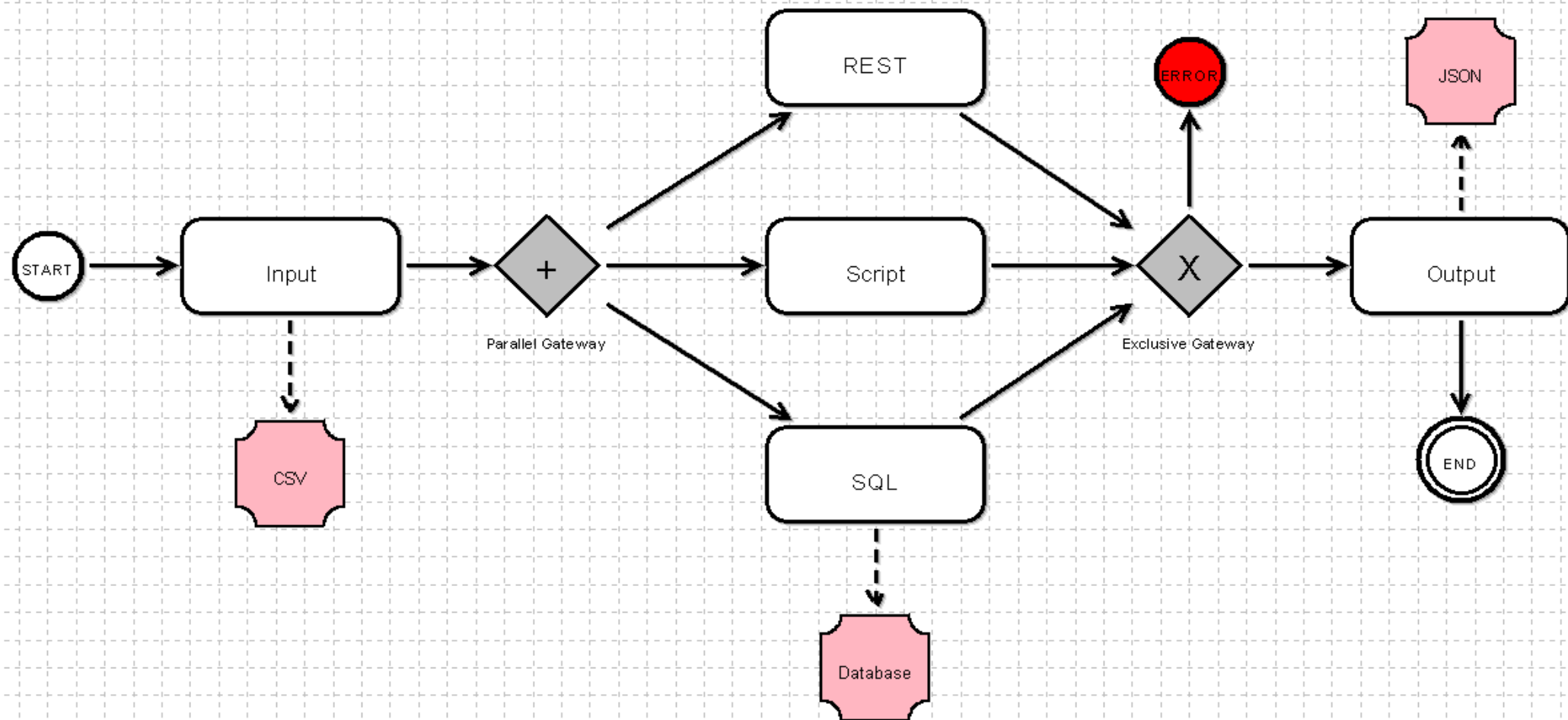
OBJECT TYPES:

```
O_TaskWeaver = {  
    Node,  
    Event,  
    Task,  
    Gateway,  
    DataObject,  
    SequenceFlow,  
    DataFlow  
}
```

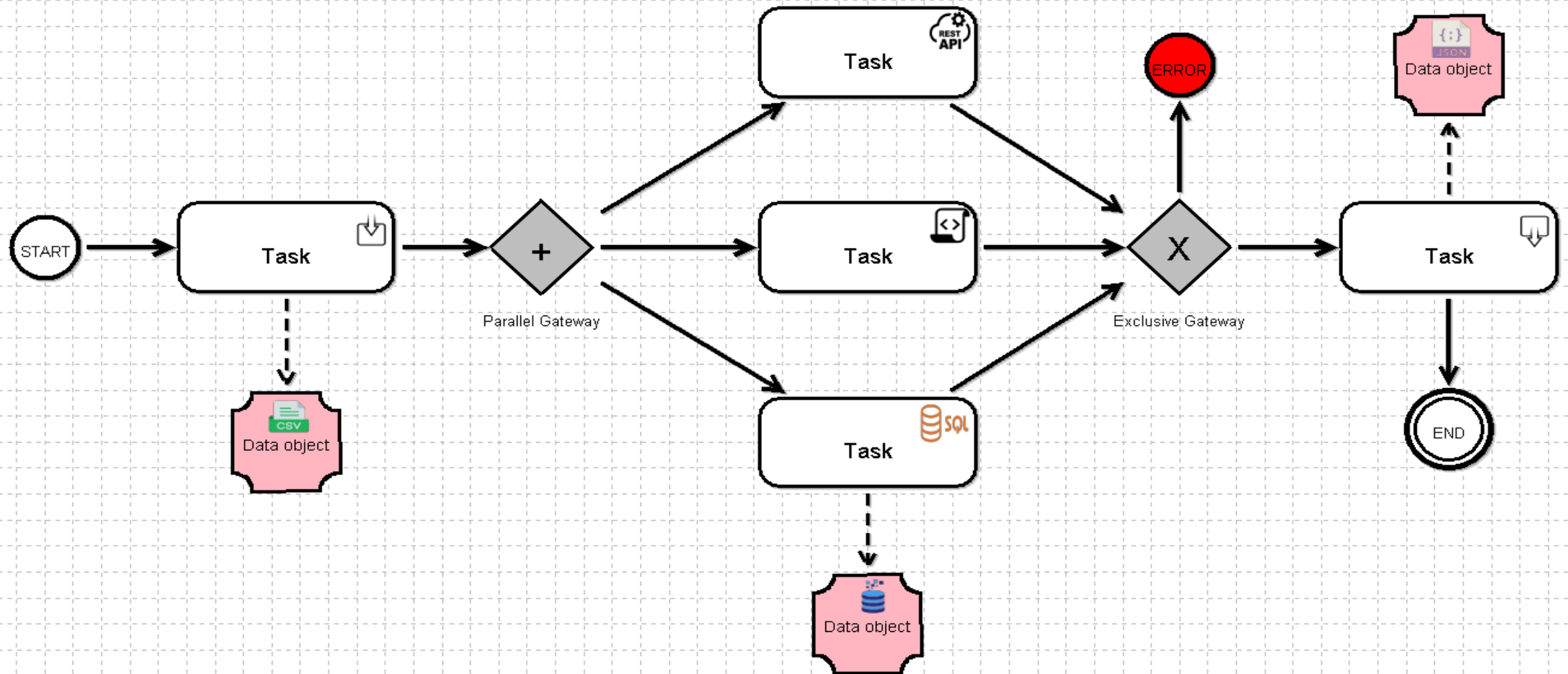
DATA TYPES:

```
D_TaskWeaver = {  
    String,  
    Double,  
    EventType,    // {Start, Error, End}  
    GatewayType,  // {Exclusive, Parallel}  
    TaskType,     // {Input, REST, SQL, Script, Output}  
    FormatType    // {CSV, JSON, Database, DataObject}  
}
```

Our First Model Instance



Improved Model Instance



Real-World Use Case Scenario

Automated Customer Data Processing Pipeline, step by step:

- > A CSV file with raw customer data is ingested (*ID, name, email, address, purchases*).
- > The workflow splits into three parallel actions:
 - REST Task: enriches data via external API
 - Script Task: validates and cleans records
 - SQL Task: stores data in the enterprise database
- > The database state is represented by a Database Data Object.
- > The workflow synchronizes at the Exclusive Gateway:
 - Failure → Error Event
 - Success → continue to output
- > The Output Task generates a verified JSON result, and the workflow terminates.



Language Architecture

What is currently implemented in ADOxx:

- Metamodel
- Attributes & Enumerations
- GraphReps
- AttrReps
- Constraints
- Task icons via BITMAP
- Flow distinctions
- (Upcoming) ADOscript Execution & Validation

+

•

○

Algorithm Execution Plan (ADOScript)

1. Model Selection & Load



2. Structural Validation

- No Event → Event connections
- Tasks must define Input & Output
- SQL Tasks must use a Database DataObject



3. Gateway Consistency Validation

- Parallel Gateway → at least 2 outgoing flows
- Exclusive Gateway → only one active outgoing path



4. Validation Decision

- Invalid → Execution Aborted
- Valid → Execution Starts



5. Task Execution (Simulated)

- REST Task → API call simulated
- Script Task → validation simulated
- SQL Task → database insert simulated

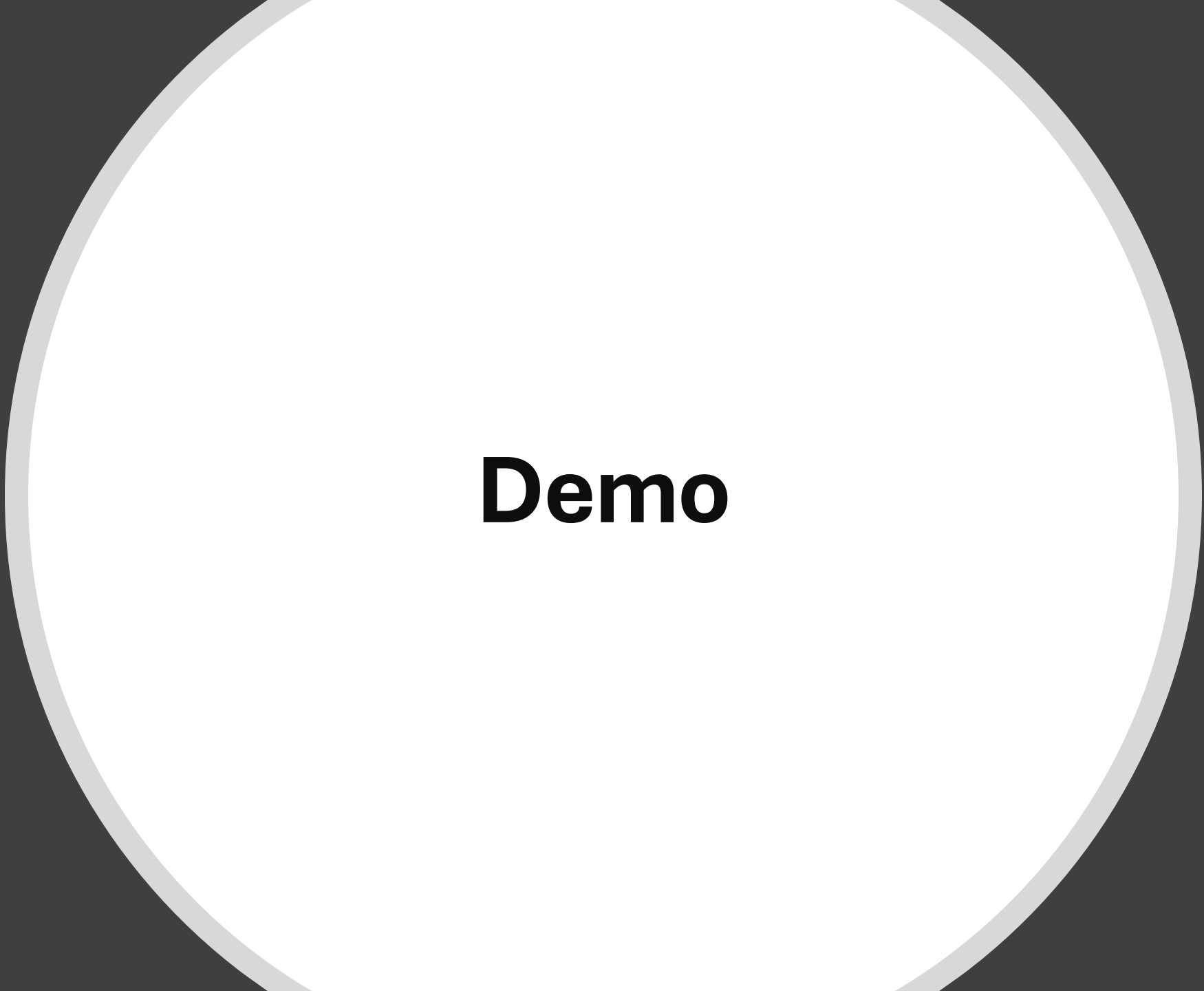


6. Output Generation

- Final JSON result produced



7. Workflow Ends



Demo



Current Status and Next Steps



What is already finished:

- Full metamodel
- Graphical notation
- FDMM formalization
- Working modeling tool
- Real-world example (model instance)

What will be delivered by Dec 17:

- ADOscript execution
 - Model validation rules
 - Final .abl and .adl files delivery
 - External data integration (potentially)
- 



Thank you.

Questions? 🤪